

# Relatório de Desenvolvimento: Engenharia e Modelagem Preditiva de Churn

**Analistas:** Pablo Henrique, Ricardo, Daniela,

**Escopo:** Pipeline End-to-End para Predição de Evasão de Clientes (Telecom)

## 1. Arquitetura de Dados e Pipeline Inicial

O projeto foi iniciado com o estabelecimento de descobrir métricas para avaliação de churn, focado em reprodutibilidade e escalabilidade.

1. **Ingestão e Sanity Check:** Carregamento de dados com tratamento de tipos e limpeza de ruídos.
2. **EDA (Exploração de Dados):** Identificamos que o maior fator de churn é por contrato mensal. O gráfico de "**Churn por Tipo de Contrato**" mostra que contratos *Month-to-month* possuem uma taxa de churn maior (~42%) comparado com a taxa de contratos anuais (~11%).
3. **Data Engineering:**
  - a. **Feature Transformation:** Aplicação de OneHotEncoder para variáveis categóricas e StandardScaler para variáveis numéricas (TotalCharges, Tenure, etc).
  - b. **Pipeline Automático:** Uso de ColumnTransformer para garantir qualidade e organização no pré-processamento, evitando *data leakage*.

## 2. Estratégia de Balanceamento de Classe

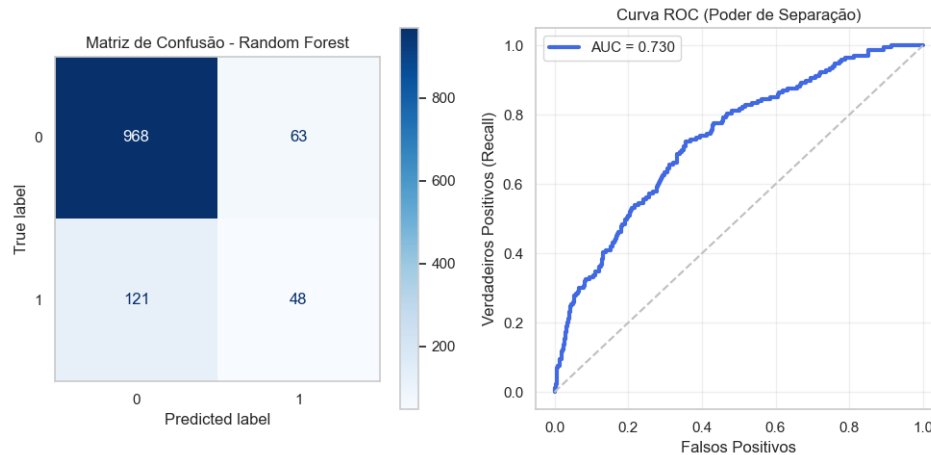
Detectamos um desequilíbrio significativo (taxa de churn x não churn). Para evitar que o modelo fosse enviesado, foi decidido aplicar o **SMOTE (Synthetic Minority Over-sampling Technique)**.

- **Decisão:** Para o modelo não ficar 'viciado' apenas nos clientes que ficam, equilibramos com novos exemplos sintéticos de churn. Buscando que o modelo não decore o comportamento da maioria.

### 3. Benchmarking de Modelos

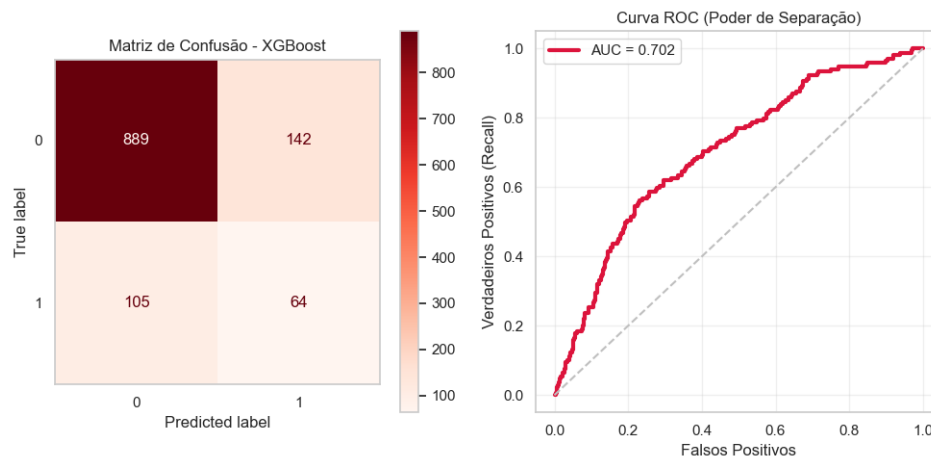
Foram implementadas três arquiteturas distintas para avaliar o comportamento dos modelos.

#### I. Random Forest (Estabilidade)



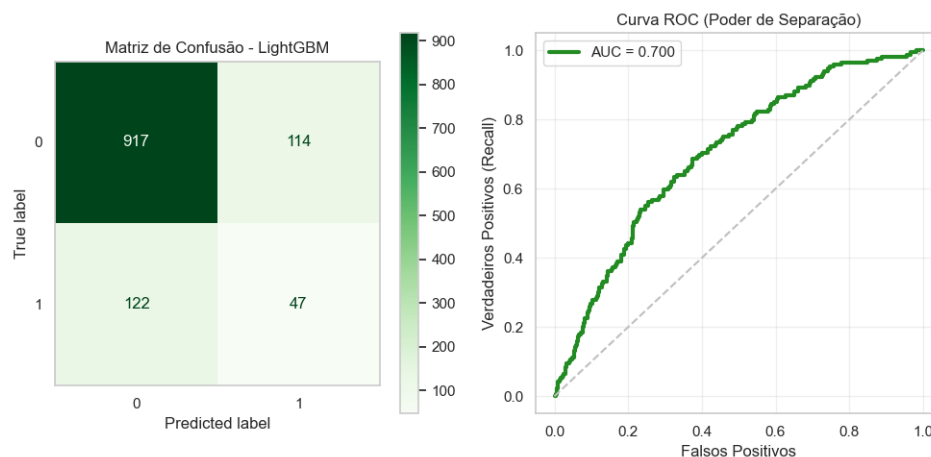
- **Comportamento:** Apresentou a maior **Acurácia (0.8467)** e **Precisão (0.4324)**.
- **Diagnóstico:** Foi o modelo mais eficiente em separar as classes (ROC-AUC de 0.7304), porém com um Recall baixo (0.2840).

#### II. XGBoost (Foco em Sensibilidade)



- **Comportamento:** Otimizado com `scale_pos_weight` para focar na classe de churn.
- **Diagnóstico:** Obteve o maior **Recall do projeto (0.3787)**. Na matriz de confusão, foi o que mais encontrou churners reais.

### III. LightGBM (Eficiência Computacional)



- **Comportamento:** Treinado com `verbosity=-1` para logs limpos e foco em performance.
- **Diagnóstico:** Desempenho equilibrado, com ROC-AUC de 0.70.

## 4. Análise de Resultados Técnicos

| Métrica  | Random Forest | XGBoost | LightGBM |
|----------|---------------|---------|----------|
| ROC-AUC  | 0.7304        | 0.7019  | 0.7000   |
| Recall   | 0.2840        | 0.3787  | 0.2781   |
| Precisão | 0.4324        | 0.3107  | 0.2919   |
| Acurácia | 0.8467        | 0.7942  | 0.8033   |

## 5. Decisão de Engenharia e Insight de Negócio

Acreditamos que o melhor modelo para implementação seja o **XGBoost**.

**Justificativa Técnica:** o custo de um Falso Negativo (perder um cliente) é exponencialmente maior que o custo de um Falso Positivo (desconto para cliente não

churn). O XGBoost lida os Falsos Negativos melhor que o Random Forest, mesmo tendo métricas globais superiores.

**Insight Principal:** A engenharia de atributos e a análise de `feature_importance` provaram que o **modelo de contratação** é o principal vetor de risco.

- **Estratégia:** Migrar clientes mensais para contratos anuais através de gatilhos automáticos de CRM baseados no *score* de probabilidade gerado pelo nosso modelo.

**Status do Projeto:** Apesar de não atingir ROC superior a 0.80, foi analisado que o limitador do modelo é o dataset de treinamento, após essas avaliações a empresa telefônica pode começar a aderir a uma melhoria na coleta de dados e históricos de comportamentos .